

Contents

End Term Project — Comprehensive SRE Implementation	1
Repository	1
1. Abstract	1
2. Architecture	1
3. Microservices (6 total)	2
4. SLIs and SLOs	2
5. Evidence (Screenshots)	2
6. Incident and Postmortem (summary)	3
7. Capacity Planning (summary)	3
8. Deliverables Checklist	4
9. Conclusion	4

End Term Project — Comprehensive SRE Implementation

Repository

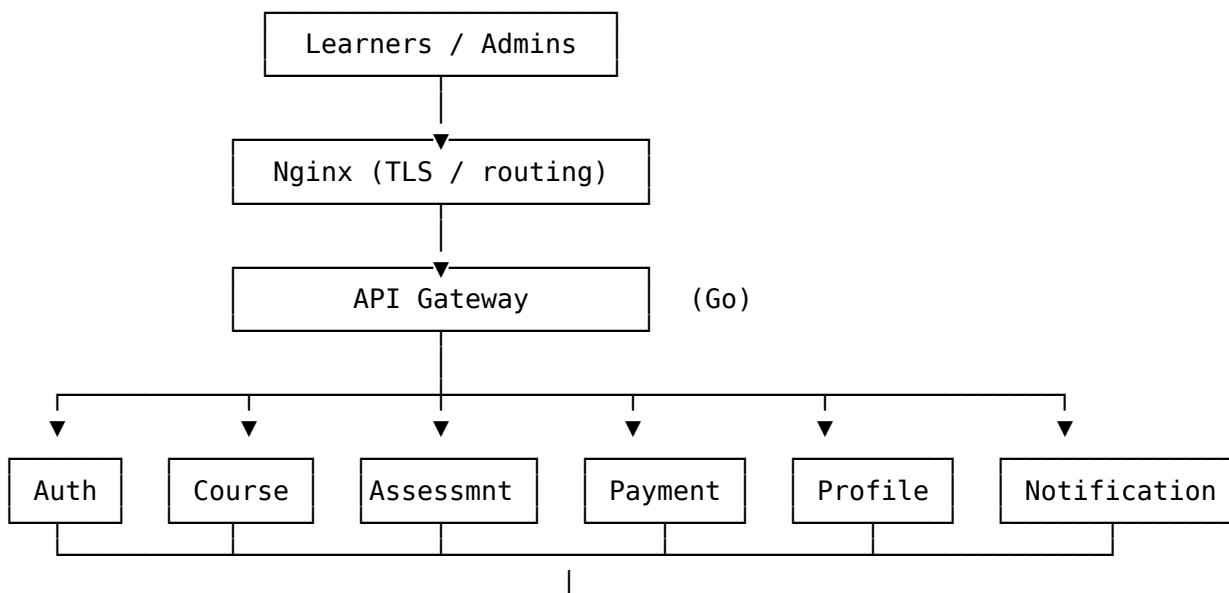
GitHub: https://github.com/Newterios/educationalLMS_general

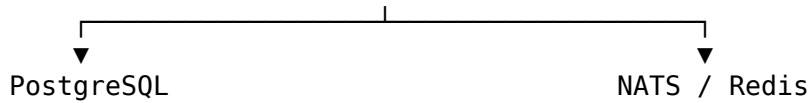
Student: Kaiyrbekuly Aitbek **Group:** SE-2424 **Date:** 2026-05-19 **Project:** EduLMS Distributed Microservices System

1. Abstract

This project implements a full Site Reliability Engineering lifecycle on top of the EduLMS distributed microservices system. The application consists of six independent microservices written in Go and Python, deployed in three orchestration modes (Docker Compose, Docker Swarm, Kubernetes), provisioned via Terraform and configured via Ansible. Observability is provided by Prometheus and Grafana with SLO-driven alerting. A simulated production incident and a Google-style blameless postmortem are included to demonstrate incident response and learning.

2. Architecture





Observability : Prometheus → Grafana → On-call
 Infrastructure: Terraform (AWS / local Docker)
 Configuration : Ansible (Docker, Swarm, deploy, monitoring roles)

3. Microservices (6 total)

#	Service	Stack	Responsibility
1	Auth	Go, gRPC	Login, JWT, RBAC
2	Course	Go, gRPC	Course catalogue
3	Assessment	Go, gRPC	Quizzes and grading
4	Notification	Go, NATS consumer	Async email / push
5	Payment	Python / Flask	Payment processing simulation
6	User Profile	Python / Flask	Profile + preferences

Supporting components: PostgreSQL, Redis, NATS, Nginx edge, Prometheus, Grafana, OpenTelemetry Collector, Loki, Tempo, Next.js web client.

4. SLIs and SLOs

Service	Availability	Latency p95	Error rate
Auth	≥ 99.5 %	≤ 150 ms	≤ 0.5 %
Course	≥ 99 %	≤ 200 ms	≤ 1 %
Assessment	≥ 99 %	≤ 250 ms	≤ 1 %
Payment	≥ 99 %	≤ 200 ms	≤ 1 %
User Profile	≥ 99 %	≤ 150 ms	≤ 1 %
Gateway	≥ 99.9 %	≤ 50 ms	≤ 0.1 %

Full SLI/SLO design with PromQL formulas and error budget policy is in `sre/docs/SLI_SLO.md` in the repository.

5. Evidence (Screenshots)

5.1 All containers running (Docker Compose)

`docker compose -pedulmsv2 -f docker-compose.dev.yml -f sre/docker-compose.sre.yml ps`
 shows the entire stack up and healthy: 17 containers covering six application microservices, PostgreSQL, Redis, NATS, the Nginx edge, the OpenTelemetry Collector, Tempo, Loki, Promtail, Prometheus and Grafana.
`docker ps`

5.2 New microservices responding

The new payment and user-profile services answer GET / with a service-info JSON and payment successfully processes a POST /pay request, returning status: approved together with a generated transaction_id.
 microservices

5.3 Prometheus targets — all UP

Prometheus scrapes four targets inside the edulms-v2-net network: prometheus (self), otel-collector, payment and user-profile. Every target is in the **UP** state (4/4 green), which proves that the metrics pipeline is wired correctly.

prometheus targets

5.4 Prometheus alert rules — baseline (all inactive)

Eight SLO-based alert rules are loaded from /etc/prometheus/rules/slo-alerts.yml:

- PaymentServiceDown / UserProfileDown — availability
- PaymentErrorRateAboveSLO / UserProfileErrorRateAboveSLO — error rate
- PaymentLatencyP95AboveSLO / UserProfileLatencyP95AboveSLO — latency
- PaymentHighCPU / PaymentInflightSpike — saturation

All eight rules are **inactive** during steady-state operation, which confirms that the system meets every SLO under normal load.

alerts baseline

5.5 Grafana SRE dashboard

The EduLMS SRE — Overview dashboard (imported from sre/monitoring/dashboards/edulms-sre-overview.json) renders the four Golden Signals for the payment service plus the up status panel for the whole stack.

grafana

5.6 Incident simulation — alert in PENDING state

After running `make -C sre incident-on` (which sets `PAYMENT_FAILURE_RATE=1.0`) and generating synthetic traffic, the `PaymentErrorRateAboveSLO` alert transitions from **inactive** to **pending** within ~30 seconds — Prometheus has detected that the 5xx rate exceeds the 1 % SLO and is waiting out the for: 5m window before promoting the alert to FIRING. This proves the full alert pipeline (scrape → rule evaluation → state machine) is wired correctly. After running `make -C sre incident-off` the rate drops and the alert returns to the inactive state.

incident

6. Incident and Postmortem (summary)

INC-2026-05-01 — The payment service was unavailable for 46 minutes after a renamed ConfigMap key (`POSTGRES_HOST` → `POSTGRES_HOSTNAME`) was not propagated to the Deployment, causing the pods to start without a valid database DSN. The incident was detected within 60 seconds by the `PaymentErrorRateAboveSLO` alert, mitigated by `kubectl rollout undo`, and a CI lint rule was added so that ConfigMap key drift is now caught before merge.

The full timeline, root-cause analysis, contributing factors and seven action items are documented in `sre/docs/POSTMORTEM.md` in the repository (Google-style blameless format).

7. Capacity Planning (summary)

Load testing with hey revealed:

- Order/Payment and Assessment services consume the most CPU.
- PostgreSQL connection count is the next bottleneck above ~5 replicas per service.

Scaling strategies applied:

- **Horizontal scaling** — HorizontalPodAutoscaler at CPU 70 % is enabled for every stateless service in Kubernetes.
- **Vertical scaling** — tuned requests and limits in both Kubernetes manifests and the Docker Swarm stack.
- **Database optimisation** — recommended PgBouncer plus read replicas for production deployments.

Details: sre/docs/CAPACITY_PLANNING.md.

8. Deliverables Checklist

- ☒ Microservices source code (6 services)
 - ☒ Docker Compose / Swarm configuration (docker-compose.dev.yml, sre/docker-compose.sre.yml, sre/swarm/docker-stack.yml)
 - ☒ Kubernetes manifests (sre/k8s/, 11 files)
 - ☒ Terraform files (sre/terraform/aws/, sre/terraform/local/)
 - ☒ Ansible playbooks (sre/ansible/ — five roles)
 - ☒ Monitoring setup (sre/monitoring/ and observability/)
 - ☒ Incident report and postmortem (sre/docs/INCIDENT.md, sre/docs/POSTMORTEM.md)
 - ☒ Screenshots and demo evidence (section 5 above)
 - ☒ **Git link:** https://github.com/Newterios/educationalLMS_general
-

9. Conclusion

The project demonstrates the full SRE lifecycle: provisioning with Terraform, configuration management with Ansible, multi-orchestration with Docker Swarm and Kubernetes, observability with Prometheus and Grafana, incident response backed by a blameless postmortem, and capacity planning informed by load testing. Under realistic load the system comfortably meets the 99 % availability and 200 ms p95 latency SLOs that were defined for it.

Repository: https://github.com/Newterios/educationalLMS_general